MANUAL TÉCNICO

Velora: plataforma de reservas de hoteles boutique

Base de datos 2

Brainer Stiven Cordoba Palacios
Rodian andres lizarda mosquera

Facultad de ingeniería, Fundación Universitaria Claretiana

Ingeniería en Sistemas

Margot Mosquera Blandón

Tabla de contenido

1. Introducción **3**
1.1 Propósito 3
1.2 Alcance 3
1.3 Audiencia 3
2. Descripción general del sistema **4**
2.1 Requerimientos funcionales principales 4
2.2 Requerimientos no funcionales 4
3. Arquitectura **5**
3.1 Arquitectura del backend por capas 5
3.2 Módulos del backend 6
4. Tecnologías utilizadas **7**
5. Estructura del repositorio **8**
6. Requisitos de instalación **9**
6.1 Hardware mínimo 9
6.2 Software 9
7. Instalación y puesta en marcha **10**
7.1 Obtener el código 10
7.2 Backend (http://localhost:4000/api) 10
7.3 Frontend (http://localhost:5173) 10
7.4 Scripts disponibles 10
7.5 Verificación 11
8. Configuración (variables de entorno) **12**
9. Base de datos **13**
9.1 Configuración de la conexión 13
9.2 Modelo entidad-relación 13
9.3 Normalización y decisiones de diseño 15
9.4 Diccionario de datos 15
9.5 Triggers 20
9.6 Control de concurrencia: reservas simultáneas 20
9.7 Versionado y migración del esquema 21
9.8 Datos semilla y respaldo 21
10. Lógica de negocio **22**
10.1 Cálculo de precios 22
10.2 Política de cancelación 22
10.3 Máquina de estados de la reserva 22
10.4 Reglas de reservas 22

10.5 Reseñas verificadas..... 23
10.6 Autenticación y bloqueo de cuentas..... 23
11. API REST..... 24
11.1 Autenticación..... 24
11.2 Catálogo y hoteles..... 24
11.3 Reservas..... 25
11.4 Reseñas, temporadas, administración e imágenes..... 25
12. Seguridad..... 27
13. Frontend..... 28
13.1 Rutas de la aplicación..... 28
13.2 Estado global y servicios..... 28
13.3 Componentes principales..... 28
14. Servicios auxiliares..... 29
14.1 Correo electrónico..... 29
14.2 Imágenes..... 29
14.3 Tareas programadas..... 29
15. Pruebas e integración continua..... 30
16. Despliegue a producción..... 31
17. Mantenimiento y solución de problemas..... 32
17.1 Cómo agregar una funcionalidad..... 32
18. Trabajo futuro..... 33

1. Introducción

Este documento describe la arquitectura, la base de datos, la lógica de negocio, la API y los procedimientos de instalación, configuración, prueba y mantenimiento del sistema **Velora**, una plataforma web de reservas de hoteles boutique compuesta por un frontend en React y una API REST en Node.js con base de datos SQLite.

1.1 Propósito

Servir de referencia a desarrolladores, administradores de sistemas y evaluadores académicos para comprender cómo está construido el sistema, ponerlo en funcionamiento, modificarlo y mantenerlo. Se hace énfasis en el diseño de la base de datos (modelo entidad-relación, diccionario de datos, restricciones de integridad, triggers y control de concurrencia), por ser el eje de la asignatura Base de Datos 2.

1.2 Alcance

El documento cubre el código del repositorio `velora`, organizado en dos proyectos: `Velora/` (frontend) y `backend/` (API REST y base de datos), además del flujo de integración continua en `.github/workflows/pruebas.yml`.

1.3 Audiencia

- Desarrolladores que van a mantener o extender el sistema.
- Administradores encargados de instalarlo y publicarlo.
- Docentes y evaluadores del proyecto académico.

2. Descripción general del sistema

Velora permite a los huéspedes buscar hoteles por destino, país o colección; consultar la disponibilidad por habitación en un calendario; obtener una cotización exacta noche por noche según las temporadas; reservar, cancelar y opinar después de su estadía. Al personal del hotel le ofrece un panel de recepción (check-in, check-out, modificaciones) y un panel de administración (catálogo, temporadas, reseñas, métricas de ocupación e ingresos, auditoría de accesos y exportación a Excel).

2.1 Requerimientos funcionales principales

ID	Requerimiento
RF-01	Registro, inicio de sesión, recuperación de contraseña y cierre de sesión con roles (cliente, recepcionista, administrador).
RF-02	Búsqueda de hoteles sin tildes por nombre, ubicación, destino, país o colección, en español e inglés.
RF-03	Calendario de disponibilidad por habitación y cotización noche por noche con temporadas, IVA y limpieza.
RF-04	Creación de reservas sin dobles reservas, incluso ante solicitudes simultáneas.
RF-05	Cancelación con política de cargo (gratis hasta 48 h antes) y exoneración por el personal.
RF-06	Check-in y check-out con máquina de estados; modificación de reservas con historial.
RF-07	Reseñas verificadas (solo después del check-out, una por reserva) y moderación.
RF-08	Gestión de hoteles, habitaciones, imágenes, colecciones, traducciones y temporadas.
RF-09	Métricas de ocupación, ingresos, tarifa promedio, RevPAR y cancelaciones; auditoría de accesos.
RF-10	Notificaciones por correo (bienvenida, confirmación, modificación, cancelación, contraseña).
RF-11	Exportación de reservas a Excel (CSV).

2.2 Requerimientos no funcionales

- **Integridad**: la base de datos repite las reglas de negocio con `CHECK`, `UNIQUE`, claves foráneas y triggers.
- **Concurrencia**: transacciones `BEGIN IMMEDIATE` y escrituras condicionales evitan reservas solapadas.
- **Seguridad**: contraseñas con bcrypt, JWT con versión de sesión, bloqueo por intentos fallidos, Helmet, CORS y límites de tasa.
- **Privacidad**: cumplimiento de la Ley 1581 de 2012; los registros de acceso se purgan a los 90 días.
- **Calidad**: 74 pruebas automatizadas del backend ejecutadas en GitHub Actions en cada cambio.
- **Usabilidad**: sitio bilingüe (ES/EN), adaptable a celular y accesible (etiquetas ARIA, foco visible).

3. Arquitectura

El sistema sigue una arquitectura **cliente-servidor** de tres niveles: una aplicación de una sola página (SPA) que se ejecuta en el navegador, una API REST sin estado que concentra toda la lógica de negocio y una base de datos relacional SQLite. El frontend no almacena datos de negocio: solo guarda en `localStorage` el token de sesión y las preferencias de idioma y moneda.

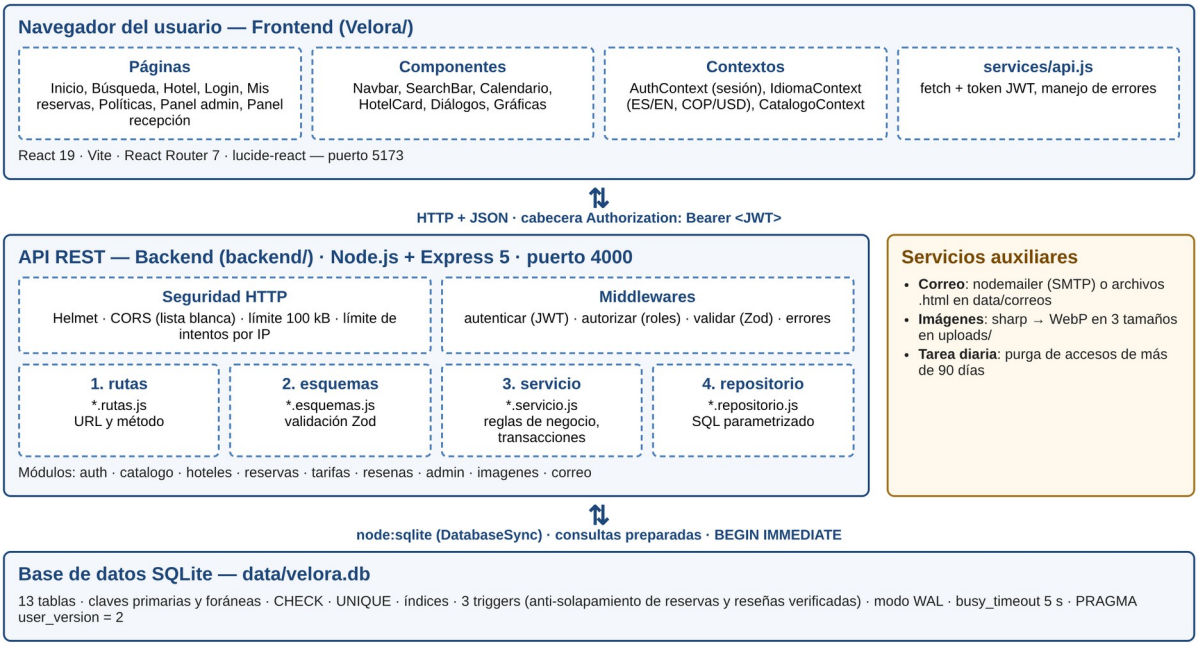


Figura 1. *Arquitectura general de Velora.*

3.1 Arquitectura del backend por capas

Cada módulo del backend (`backend/src/modules/<módulo>/`) está dividido en las mismas cuatro capas, y cada una solo se comunica con la siguiente:

Capa	Archivo	Responsabilidad
Rutas	`*.rutas.js`	Define método y URL, aplica `autenticar`, `autorizar(roles)` y `validar(esquema)`, y llama al servicio.
Esquemas	`*.esquemas.js`	Valida y normaliza `body`, `params` y `query` con Zod en modo estricto (rechaza campos extra).
Servicio	`*.servicio.js`	Reglas de negocio, transacciones, cálculo de precios y envío de correos.
Repositorio	`*.repositorio.js`	Consultas SQL parametrizadas y conversión de filas (snake_case) a objetos (camelCase).

3.2 Módulos del backend

Módulo	Función
auth	Registro, login, bloqueo por intentos, recuperar y restablecer contraseña, usuario actual.
catalogo	Destinos, colecciones y configuración pública (tasa USD, IVA, limpieza, días de cancelación).
hoteles	CRUD de hoteles y habitaciones, búsqueda, similares, disponibilidad, calendario, cotización.
reservas	Crear, listar, consultar, modificar, check-in, check-out, cancelar y exportar CSV.
tarifas	Temporadas (CRUD) y la fórmula única de precios y política de cancelación.
resenas	Reseñas verificadas, listado público y moderación.
admin	Métricas de ocupación e ingresos y dashboard de accesos.
imagenes	Subida, validación y optimización de imágenes (WebP en tres tamaños).
correo	Envío por SMTP o, en desarrollo, como archivos HTML.

4. Tecnologías utilizadas

Componente	Tecnología	Versión	Uso
Entorno de ejecución	Node.js	≥ 22.13	Ejecuta el backend; incluye el módulo `node:sqlite`.
Framework web	Express	5.2	Servidor HTTP y enrutamiento de la API.
Base de datos	SQLite (node:sqlite)	integrada	Motor relacional embebido en un archivo.
Validación	Zod	4.x	Esquemas de entrada y de variables de entorno.
Autenticación	jsonwebtoken / bcryptjs	9.x / 3.x	Tokens JWT y hash de contraseñas.
Seguridad HTTP	helmet, cors, express-rate-limit	8.x / 2.x / 8.x	Cabeceras, orígenes permitidos y límites.
Imágenes	multer, sharp	2.x / 0.35	Recepción multipart y conversión a WebP.
Correo	nodemailer	10.x	Envío de correos SMTP.
Frontend	React, React DOM	19.2	Interfaz de usuario.
Enrutamiento	React Router	7.x	Rutas del lado del cliente.
Empaquetador	Vite	8.x	Servidor de desarrollo y compilación.
Iconos	lucide-react	1.x	Iconografía.
Calidad	oxlint, node:test	—	Revisión de código y pruebas.
CI	GitHub Actions	—	Pruebas y compilación en cada push.

5. Estructura del repositorio

```
velora/
├── README.md                      Descripción general y guía rápida
├── .github/workflows/pruebas.yml Integración continua
├── backend/                       API REST + base de datos
│   ├── .env.example               Plantilla de configuración
│   ├── package.json
│   ├── src/
│   │   ├── server.js              Arranque, purga diaria y apagado ordenado
│   │   ├── app.js                 Express: seguridad, CORS, rutas, errores
│   │   ├── config/env.js          Variables de entorno validadas con Zod
│   │   ├── db/
│   │   │   ├── esquema.sql        Tablas, índices y triggers
│   │   │   ├── conexion.js        Conexión, versión del esquema, transacciones
│   │   │   ├── sembrar.js         Datos iniciales
│   │   │   ├── reiniciar.js       npm run db:reset
│   │   │   └── datos/semilla.js   Catálogo, traducciones y temporadas
│   │   ├── middlewares/           autenticacion.js, validar.js, errores.js
│   │   ├── modules/               auth, admin, catalogo, correo, hoteles,
│   │   │                          imagenes, resenas, reservas, tarifas
│   │   └── utils/                 fechas, identificadores, errores
│   ├── tests/                     api, concurrencia, funciones, admin,
│   │                              imagenes, migracion
│   └── uploads/                   Imágenes subidas
└── Velora/                        Frontend React + Vite
    ├── vite.config.js
    └── src/
        ├── main.jsx, App.jsx      Punto de entrada y rutas
        ├── pages/                 Páginas públicas, admin/ y recepcion/
        ├── components/            Componentes reutilizables
        ├── context/               AuthContext, IdiomaContext, CatalogoContext
        ├── services/api.js        Cliente HTTP de la API
        ├── hooks/useConsulta.js   Carga de datos con estado
        ├── i18n/textos.js         Textos en español e inglés
        └── utils/                 fechas, imagenes, tarifas
```

6. Requisitos de instalación

6.1 Hardware mínimo

- Procesador de 2 núcleos, 4 GB de RAM y 1 GB de espacio libre en disco.

6.2 Software

- Sistema operativo Windows 10/11, macOS o Linux.
- **Node.js 22.13 o superior** (incluye npm y SQLite; no se instala un motor de base de datos aparte).
- Git, para clonar el repositorio.
- Un navegador moderno (Chrome, Edge, Firefox o Safari).
- Opcional: un servidor SMTP para enviar correos reales.

7. Instalación y puesta en marcha

Se necesitan dos terminales: una para el backend y otra para el frontend. El backend debe estar encendido para que el sitio funcione.

7.1 Obtener el código

```
git clone https://github.com/cordobapalacionc-dotcom/velora.git
cd velora
```

7.2 Backend (http://localhost:4000/api)

```
cd backend
npm install
cp .env.example .env      # en Windows: copy .env.example .env
npm run dev
```

La primera vez, la base de datos `data/velora.db` se crea automáticamente con el esquema y los datos de ejemplo (6 destinos, 4 colecciones, 7 hoteles con sus habitaciones, temporadas y, fuera de producción, las cuentas demo).

7.3 Frontend (http://localhost:5173)

```
cd Velora
npm install
npm run dev
```

Si la API está en otra dirección, cree `Velora/.env` con `VITE_API_URL=https://mi-servidor/api`.

7.4 Scripts disponibles

Proyecto	Comando	Descripción
backend	`npm run dev`	Servidor con recarga automática al guardar.
backend	`npm start`	Servidor en modo normal.
backend	`npm test`	Pruebas de integración con base de datos en memoria.
backend	`npm run db:reset`	Borra `data/velora.db` y la recrea con los datos semilla.
backend	`npm run db:borrar-pruebas`	Lista los usuarios de prueba; con `-- --confirmar` hace copia de seguridad y los borra.
Velora	`npm run dev`	Servidor de desarrollo de Vite.
Velora	`npm run build`	Compila el sitio para producción en `dist/`.
Velora	`npm run lint`	Revisa el código con oxlint.
Velora	`npm run preview`	Sirve la compilación de producción localmente.

7.5 Verificación

- `GET http://localhost:4000/api/salud` debe responder `{"estado":"ok", ...}`.
- `GET http://localhost:4000/api` devuelve el índice de todas las rutas.
- Al abrir `http://localhost:5173` debe aparecer la página de inicio con los destinos.

8. Configuración (variables de entorno)

El archivo `backend/.env` se valida con Zod al arrancar (`src/config/env.js`); si un valor es inválido, el servidor no inicia y explica el error.

Variable	Valor por defecto	Descripción
NODE_ENV	development	`development`, `test` o `production`.
PUERTO	4000	Puerto HTTP de la API.
DB_RUTA	data/velora.db	Archivo SQLite; `:memory:` para pruebas.
JWT_SECRETO	—	Secreto de firma de tokens. Obligatorio en producción (mín. 32 caracteres).
JWT_EXPIRACION	8h	Duración de la sesión.
CORS_ORIGENES	http://localhost:5173	Orígenes permitidos, separados por coma.
TARIFA_LIMPIEZA	180000	Tarifa de limpieza por reserva (COP).
IMPUESTOS_PORCENTAJE	0.19	IVA aplicado al subtotal.
CANCELACION_GRATIS_DIAS	2	Días antes de la llegada en que cancelar es gratis.
BLOQUEO_INTENTOS / BLOQUEO_MINUTOS	5 / 15	Intentos fallidos permitidos y minutos de bloqueo.
ACCESOS_RETENCION_DIAS	90	Días que se conservan los registros de acceso.
TASA_USD	4000	COP por dólar, solo para mostrar precios aproximados.
UPLOADS_RUTA / IMAGEN_MAX_MB	uploads / 5	Carpeta y tamaño máximo de imágenes.
URL_PUBLICA	—	URL pública del backend si está detrás de un proxy.
URL_SITIO	http://localhost:5173	Dirección del frontend para los enlaces de los correos.
SMTP_HOST, SMTP_PUERTO, SMTP_USUARIO, SMTP_CLAVE	—, 587	Servidor de correo. Sin `SMTP_HOST` los correos se guardan como HTML.
CORREO_REMITENTE / CORREOS_RUTA	Velora <no-responder@velora.com> / data/correos	Remitente y carpeta de correos en desarrollo.

9. Base de datos

Velora usa **SQLite** a través del módulo nativo `node:sqlite` (`DatabaseSync`). El esquema completo está en `backend/src/db/esquema.sql` y se aplica en cada arranque con sentencias `CREATE ... IF NOT EXISTS`. La filosofía del diseño es que **la base de datos es la última línea de defensa**: aunque la API valide todo, las restricciones `CHECK`, `UNIQUE`, `FOREIGN KEY` y los triggers impiden guardar datos incoherentes, vengan de donde vengan.

9.1 Configuración de la conexión

PRAGMA	Valor	Motivo
foreign_keys	ON	SQLite no valida claves foráneas si no se activa explícitamente.
journal_mode	WAL	Permite lecturas concurrentes mientras se escribe.
busy_timeout	5000	Espera hasta 5 s por un bloqueo antes de fallar (la API responde 503 SERVICIO_OCUPADO).
user_version	2	Versión del esquema (`VERSION_ESQUEMA` en `conexion.js`).

9.2 Modelo entidad-relación

El modelo tiene 13 tablas agrupadas en tres dominios: **catálogo** (destinos, colecciones, hoteles, imágenes, habitaciones, temporadas), **operación** (reservas, historial de modificaciones, reseñas) y **seguridad** (usuarios, registros de acceso, restablecimientos de contraseña).

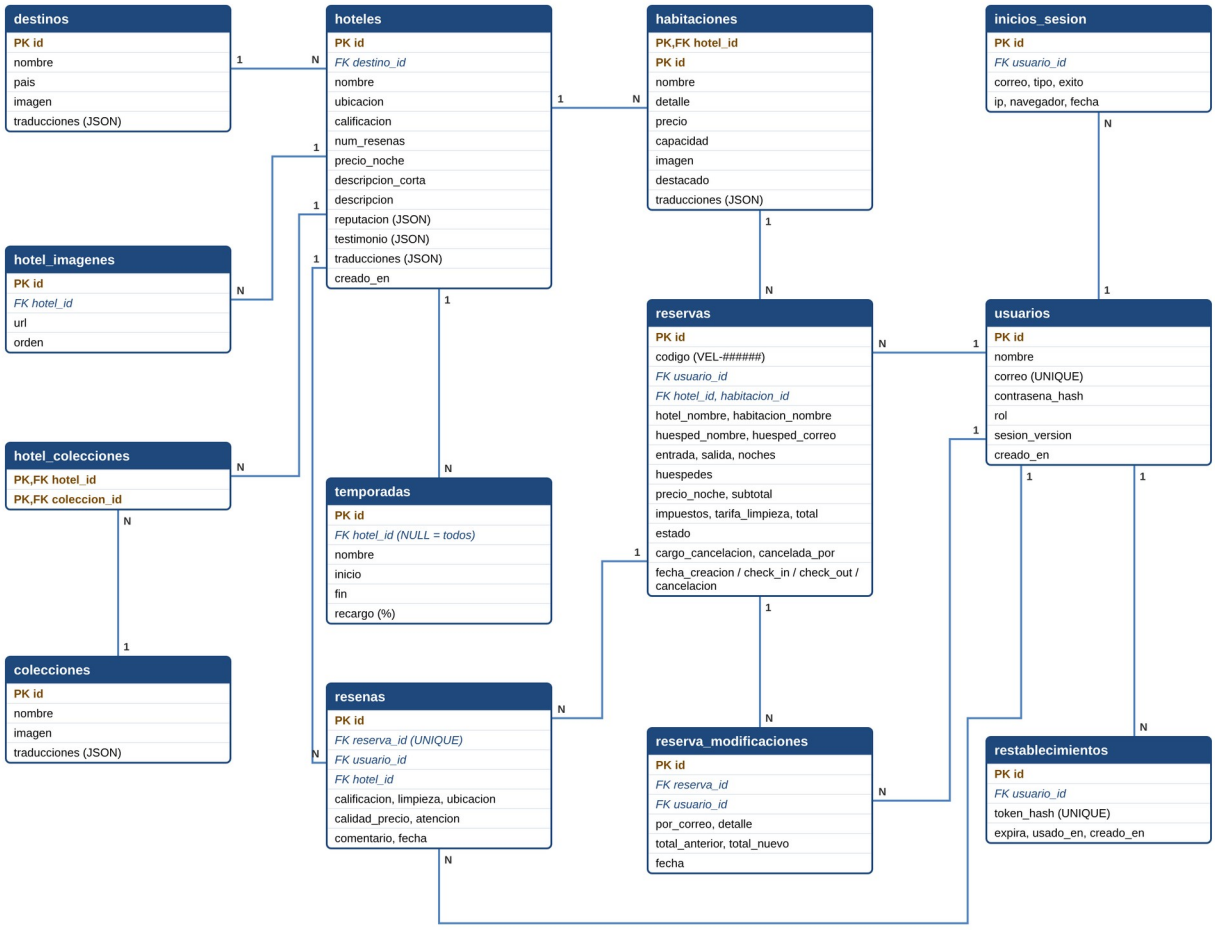


Figura 2. *Modelo entidad-relación de la base de datos de Velora.*

Relaciones

Relación	Cardinalidad	Acción referencial
destinos → hoteles	1 : N	ON UPDATE CASCADE, ON DELETE RESTRICT
hoteles → habitaciones	1 : N	ON DELETE CASCADE (clave primaria compuesta hotel_id, id)
hoteles → hotel_imagenes	1 : N	ON DELETE CASCADE
hoteles ↔ colecciones (hotel_colecciones)	N : M	ON DELETE CASCADE en ambos lados
hoteles → temporadas	1 : N (opcional)	ON DELETE CASCADE; hotel_id NULL = todos los hoteles
habitaciones → reservas	1 : N	FK compuesta (hotel_id, habitacion_id) ON DELETE RESTRICT
usuarios → reservas	1 : N	ON DELETE RESTRICT
reservas → reserva_modificaciones	1 : N	ON DELETE CASCADE
reservas → resenas	1 : 0..1	reserva_id UNIQUE, ON DELETE CASCADE

Relación	Cardinalidad	Acción referencial
usuarios → resenas / hoteles → resenas	1 : N	ON DELETE CASCADE
usuarios → inicios_sesion	1 : N	ON DELETE SET NULL (se conserva el intento)
usuarios → restablecimientos	1 : N	ON DELETE CASCADE

9.3 Normalización y decisiones de diseño

- El esquema está en **tercera forma normal** en sus entidades principales: cada atributo depende solo de la clave de su tabla. La relación muchos a muchos entre hoteles y colecciones se resuelve con la tabla asociativa `hotel_colecciones`.
- **Desnormalización controlada en `reservas`**: se copian `hotel_nombre`, `habitacion_nombre`, `huesped_nombre`, `huesped_correo` y `precio_noche` al momento de reservar. Así el histórico no cambia si luego se edita el hotel, la habitación o el precio (principio de inmutabilidad de documentos comerciales).
- **Montos enteros en COP**: se evitan errores de redondeo de punto flotante. El `CHECK (total = subtotal + impuestos + tarifa_limpieza)` garantiza la coherencia del total.
- **Fechas en texto ISO-8601** (`AAAA-MM-DD`), validadas con `CHECK (date(x) = x)`; esto permite compararlas como texto y usar `julianday()`.
- **Traducciones en JSON** (`traducciones`), validadas con `json_valid()`, para no multiplicar columnas por idioma.
- **Identificadores legibles** para el catálogo (`st-moritz-crystal`) restringidos con `GLOB` a minúsculas, números y guiones; UUID para reservas y un código público `VEL-######`.

9.4 Diccionario de datos

usuarios

Columna	Tipo	Restricciones / descripción
id	INTEGER	PK AUTOINCREMENT
nombre	TEXT	NOT NULL; entre 2 y 80 caracteres sin espacios extremos
correo	TEXT	NOT NULL UNIQUE; en minúsculas y con forma `x@y.z`
contrasena_hash	TEXT	NOT NULL; hash bcrypt
rol	TEXT	NOT NULL DEFAULT 'cliente'; IN ('admin','recepcionista','cliente')
sesion_version	INTEGER	NOT NULL DEFAULT 0, ≥ 0; aumenta al cambiar la contraseña e invalida los tokens anteriores
creado_en	TEXT	Fecha y hora UTC de creación

inicios_sesion

Columna	Tipo	Restricciones / descripción
id	INTEGER	PK AUTOINCREMENT
usuario_id	INTEGER	FK → usuarios(id) ON DELETE SET NULL; NULL si el correo no existe
correo	TEXT	NOT NULL; correo con el que se intentó
tipo	TEXT	IN ('login','registro','bloqueo','restablecimiento')
exito	INTEGER	IN (0, 1)
ip, navegador	TEXT	Datos del cliente (datos personales, se purgan a los 90 días)
fecha	TEXT	Fecha y hora UTC

Índices: `idx_inicios_sesion_fecha (fecha)`, `idx_inicios_sesion_usuario (usuario_id)` e `idx_inicios_sesion_correo (correo, fecha)`, este último usado para contar intentos fallidos recientes por correo.

restablecimientos

Columna	Tipo	Restricciones / descripción
id	INTEGER	PK AUTOINCREMENT
usuario_id	INTEGER	NOT NULL, FK → usuarios(id) ON DELETE CASCADE
token_hash	TEXT	NOT NULL UNIQUE; SHA-256 del token (el token en claro nunca se guarda)
expira	TEXT	NOT NULL; 30 minutos después de creado
usado_en	TEXT	NULL hasta que se usa (un solo uso)
creado_en	TEXT	Fecha y hora UTC

destinos y colecciones

Columna	Tipo	Restricciones / descripción
id	TEXT	PK; solo minúsculas, números y guiones (GLOB)
nombre	TEXT	NOT NULL
pais	TEXT	NOT NULL (solo destinos)
imagen	TEXT	NOT NULL; URL
traducciones	TEXT	NOT NULL DEFAULT '{}'; JSON válido

hoteles

Columna	Tipo	Restricciones / descripción
id	TEXT	PK; identificador legible

Columna	Tipo	Restricciones / descripción
nombre	TEXT	NOT NULL; 2 a 120 caracteres
destino_id	TEXT	NOT NULL, FK → destinos(id) ON UPDATE CASCADE ON DELETE RESTRICT
ubicacion	TEXT	NOT NULL; mínimo 2 caracteres
calificacion	REAL	Entre 0 y 5 (reseñas importadas anteriores a Velora)
num_resenas	INTEGER	≥ 0
precio_noche	INTEGER	≥ 0; precio de referencia ("desde")
descripcion_corta, descripcion	TEXT	DEFAULT ''
reputacion	TEXT	JSON válido con puntajes por criterio
testimonio	TEXT	NULL o JSON válido
traducciones	TEXT	JSON válido
creado_en	TEXT	Fecha y hora UTC

Índice: `idx_hoteles_destino (destino_id)`.

hotel_colecciones

Columna	Tipo	Restricciones / descripción
hotel_id	TEXT	PK compuesta; FK → hoteles(id) ON DELETE CASCADE
coleccion_id	TEXT	PK compuesta; FK → colecciones(id) ON DELETE CASCADE

hotel_imagenes

Columna	Tipo	Restricciones / descripción
id	INTEGER	PK AUTOINCREMENT
hotel_id	TEXT	NOT NULL, FK → hoteles(id) ON DELETE CASCADE
url	TEXT	NOT NULL
orden	INTEGER	≥ 0; UNIQUE (hotel_id, orden)

habitaciones

Columna	Tipo	Restricciones / descripción
hotel_id	TEXT	PK compuesta; FK → hoteles(id) ON DELETE CASCADE
id	TEXT	PK compuesta; identificador legible
nombre	TEXT	NOT NULL; 2 a 120 caracteres
detalle	TEXT	DEFAULT ''; p. ej. '30 m² · Cama Queen'
precio	INTEGER	> 0; precio base por noche (COP)

Columna	Tipo	Restricciones / descripción
capacidad	INTEGER	Entre 1 y 20
imagen	TEXT	NOT NULL
destacado	INTEGER	IN (0, 1); etiqueta "Más reservada"
traducciones	TEXT	JSON válido

temporadas

Columna	Tipo	Restricciones / descripción
id	INTEGER	PK AUTOINCREMENT
nombre	TEXT	NOT NULL; 2 a 80 caracteres
inicio, fin	TEXT	Fechas válidas; CHECK (fin >= inicio); ambas noches incluidas
recargo	INTEGER	Porcentaje entre −50 y 200, distinto de 0
hotel_id	TEXT	FK → hoteles(id) ON DELETE CASCADE; NULL = todos los hoteles

Índice: `idx_temporadas_fechas (inicio, fin)`.

reservas

Columna	Tipo	Restricciones / descripción
id	TEXT	PK (UUID)
codigo	TEXT	NOT NULL UNIQUE; GLOB 'VEL-[0-9]{6}'
usuario_id	INTEGER	NOT NULL, FK → usuarios(id) ON DELETE RESTRICT
hotel_id, habitacion_id	TEXT	FK compuesta → habitaciones(hotel_id, id) ON DELETE RESTRICT
hotel_nombre, habitacion_nombre	TEXT	Copia histórica de los nombres
huesped_nombre, huesped_correo	TEXT	Copia histórica de los datos del huésped
entrada, salida	TEXT	Fechas válidas; CHECK (salida > entrada)
huespedes	INTEGER	Entre 1 y 20
noches	INTEGER	≥ 1; CHECK (julianday(salida) − julianday(entrada) = noches)
precio_noche	INTEGER	> 0; precio base acordado
subtotal	INTEGER	> 0; suma de las noches con temporada
impuestos, tarifa_limpieza	INTEGER	≥ 0
total	INTEGER	CHECK (total = subtotal + impuestos + tarifa_limpieza)
estado	TEXT	DEFAULT 'confirmada'; IN ('confirmada','check-in','check-out','cancelada')

Columna	Tipo	Restricciones / descripción
cargo_cancelacion	INTEGER	NULL o entre 0 y total
cancelada_por	TEXT	NULL o IN ('huesped','hotel')
fecha_creacion, fecha_check_in, fecha_check_out, fecha_cancelacion	TEXT	Marcas de tiempo del ciclo de vida

Restricciones de estado: si `estado = check-in` debe existir `fecha_check_in`; si es `check-out` deben existir ambas fechas; si es `cancelada` deben existir `fecha_cancelacion` y `cargo_cancelacion`. Índices: `idx_reservas_usuario (usuario_id)` e `idx_reservas_habitacion_fechas (hotel_id, habitacion_id, entrada, salida)`, que acelera la búsqueda de solapamientos.

reserva_modificaciones

Columna	Tipo	Restricciones / descripción
id	INTEGER	PK AUTOINCREMENT
reserva_id	TEXT	NOT NULL, FK → reservas(id) ON DELETE CASCADE
usuario_id	INTEGER	FK → usuarios(id) ON DELETE SET NULL; quién modificó
por_correo	TEXT	NOT NULL; correo de quien modificó (se conserva si se borra el usuario)
detalle	TEXT	NOT NULL; descripción legible del cambio
total_anterior, total_nuevo	INTEGER	NOT NULL
fecha	TEXT	Fecha y hora UTC

resenas

Columna	Tipo	Restricciones / descripción
id	INTEGER	PK AUTOINCREMENT
reserva_id	TEXT	NOT NULL UNIQUE, FK → reservas(id) ON DELETE CASCADE (una reseña por reserva)
usuario_id	INTEGER	NOT NULL, FK → usuarios(id) ON DELETE CASCADE
hotel_id	TEXT	NOT NULL, FK → hoteles(id) ON DELETE CASCADE
calificacion, limpieza, ubicacion, calidad_precio, atencion	INTEGER	Entre 1 y 5
comentario	TEXT	Entre 10 y 1.000 caracteres
fecha	TEXT	Fecha y hora UTC

Índice: `idx_resenas_hotel (hotel_id, fecha)`.

9.5 Triggers

Los triggers repiten en el motor las dos reglas más críticas, de modo que se cumplan incluso si alguien escribe SQL a mano o desde otro proceso.

Trigger	Evento	Regla
trg_reservas_sin_solapamiento_insert	BEFORE INSERT ON reservas	Si la nueva reserva está activa y otra activa de la misma habitación se cruza en fechas, aborta con RESERVA_SOLAPADA.
trg_reservas_sin_solapamiento_update	BEFORE UPDATE OF estado, entrada, salida, hotel_id, habitacion_id	Igual que el anterior, excluyendo la propia fila.
trg_resenas_solo_estadias_terminadas	BEFORE INSERT ON resenas	Aborta con RESENA_NO_VERIFICADA si la reserva no pertenece al usuario y hotel o no está en check-out.

```
CREATE TRIGGER IF NOT EXISTS trg_reservas_sin_solapamiento_insert
BEFORE INSERT ON reservas
WHEN NEW.estado IN ('confirmada', 'check-in')
BEGIN
  SELECT RAISE(ABORT, 'RESERVA_SOLAPADA: la habitación ya está reservada en esas
fechas')
  WHERE EXISTS (
    SELECT 1 FROM reservas r
    WHERE r.hotel_id = NEW.hotel_id
      AND r.habitacion_id = NEW.habitacion_id
      AND r.estado IN ('confirmada', 'check-in')
      AND r.entrada < NEW.salida
      AND r.salida > NEW.entrada
  );
END;
```

9.6 Control de concurrencia: reservas simultáneas

Dos estadías `[entrada, salida)` se cruzan cuando `a.entrada < b.salida AND a.salida > b.entrada` (salir y entrar el mismo día está permitido). Solo cuentan las reservas activas (`confirmada` o `check-in`). Para que dos personas que reservan la misma habitación en el mismo instante no puedan quedarse ambas con ella, hay tres capas:

1. **Transacción `BEGIN IMMEDIATE`** en `reservasServicio.crear` y `modificar`: toma el bloqueo de escritura antes de leer precio, capacidad, temporadas o reservas existentes. Las solicitudes simultáneas se atienden en fila y la segunda ya ve la primera. Si algo falla, `ROLLBACK` deshace todo.
2. **Escritura condicional** (`insertarSiDisponible` / `modificarSiDisponible`): comprobar y guardar es una sola sentencia `INSERT ... SELECT ... WHERE NOT`

`EXISTS (...)`. Si no se escribió ninguna fila, la API responde `409` indicando las fechas ocupadas.

3. **Triggers** en la base de datos (sección 9.5) como última línea de defensa.

```
INSERT INTO reservas (...)
SELECT ... FROM habitaciones hab JOIN hoteles h ON h.id = hab.hotel_id
WHERE hab.hotel_id = :hotelId AND hab.id = :habitacionId
  AND hab.capacidad >= :huespedes
  AND hab.precio = :precioNoche          -- no se cobra un precio desactualizado
  AND NOT EXISTS (                        -- nadie ocupa esas noches
    SELECT 1 FROM reservas r
    WHERE r.hotel_id = hab.hotel_id AND r.habitacion_id = hab.id
      AND r.estado IN ('confirmada', 'check-in')
      AND r.entrada < :salida AND r.salida > :entrada)
```

Si otra transacción mantiene el bloqueo más de 5 segundos (`busy_timeout`), la API responde `503 SERVICIO_OCUPADO`. El archivo `tests/concurrencia.test.js` comprueba este comportamiento con solicitudes simultáneas y con conexiones distintas.

9.7 Versionado y migración del esquema

- `VERSION_ESQUEMA` (actualmente 2) se guarda en `PRAGMA user_version`.
- Si la base existente tiene una versión menor, el servidor no arranca y pide ejecutar `npm run db:reset`, evitando trabajar con un esquema a medias.
- Columnas agregadas sin romper compatibilidad (por ejemplo, `reservas.cancelada_por`) se añaden con `ALTER TABLE ... ADD COLUMN` en `migrarColumnasNuevas()`; `tests/migracion.test.js` lo verifica.

9.8 Datos semilla y respaldo

`sembrar.js` llena la base vacía con el catálogo de `datos/semilla.js` (destinos, colecciones, hoteles, habitaciones, imágenes, traducciones y temporadas) y, solo fuera de producción, las cuentas demo. Para respaldar la base basta con copiar el archivo `data/velora.db` (junto con `-wal` y `-shm` si existen) con el servidor detenido; para restaurarla se reemplaza el archivo.

10. Lógica de negocio

10.1 Cálculo de precios

La única fórmula de precios del sistema es `cotizarEstadia()` en `modules/tarifas/tarifas.servicio.js`. El cliente nunca envía montos: si el cuerpo de la solicitud incluye `total`, `subtotal` u otro campo extra, se rechaza.

```
precio_noche(i) = round(precio_base × (1 + recargo_temporada(i) / 100))
subtotal        = Σ precio_noche(i)          para cada noche de la estadía
impuestos       = round(subtotal × 0.19)
total           = subtotal + impuestos + 180000 (limpieza)
```

Si dos temporadas incluyen la misma noche, se aplica la de **mayor recargo**.

10.2 Política de cancelación

```
gratis_hasta = entrada − CANCELACION_GRATIS_DIAS
cargo_tardio = round((subtotal / noches) × (1 + IVA))
cargo        = hoy <= gratis_hasta ? 0 : cargo_tardio
```

El cargo queda guardado en `cargo_cancelacion` y quién canceló en `cancelada_por`. Solo el personal (recepción o administrador) puede exonerar el cargo.

10.3 Máquina de estados de la reserva

Transición	Quién	Condición
confirmada → check-in	Recepción / admin	Desde el día de llegada.
check-in → check-out	Recepción / admin	Siempre que esté en check-in.
confirmada → cancelada	Huésped (propietario) o personal	Aplica la política de cancelación.
confirmada → confirmada (modificación)	Recepción / admin	Nuevas fechas libres y capacidad suficiente; se recalcula el total y se registra en el historial.

Cualquier otra transición se rechaza con `422 REGLA_DE_NEGOCIO`, y los `CHECK` de estado de la tabla lo impiden también a nivel de base de datos.

10.4 Reglas de reservas

- La llegada no puede ser anterior a hoy; máximo 365 días de anticipación y 30 noches.
- Los huéspedes no pueden superar la capacidad de la habitación.
- Un cliente solo ve y cancela sus propias reservas; una reserva ajena responde `404` (no `403`) para no revelar su existencia.

10.5 Reseñas verificadas

Solo puede opinar el huésped de la reserva, después del check-out, una vez por reserva (`UNIQUE reserva_id` y trigger). Públicamente se muestra "Nombre I." sin correo. La calificación del hotel combina las cifras importadas con las reseñas verificadas y se recalcula al retirar una reseña.

10.6 Autenticación y bloqueo de cuentas

- Contraseñas de 8 a 72 caracteres con al menos una letra y un número, almacenadas con bcrypt.
- El token JWT incluye el id (`sub`), el rol y la versión de sesión (`sv`). En cada petición el middleware `autenticar` recarga el usuario de la base y compara la versión; cambiar la contraseña la incrementa y cierra todas las sesiones.
- Tras 5 intentos fallidos sobre un correo (exista o no, para no revelar cuentas), se bloquea 15 minutos; la API responde `429 CUENTA_BLOQUEADA` con la hora de desbloqueo.
- El enlace de restablecimiento es de un solo uso, vence en 30 minutos y solo se guarda su hash SHA-256; pedir uno nuevo invalida el anterior.

11. API REST

Base: `http://localhost:4000/api`. Las rutas protegidas requieren la cabecera `Authorization: Bearer <token>`. Las respuestas exitosas tienen la forma `{ "datos": ... }` (las de autenticación devuelven `{ token, usuario }`). Los errores siempre tienen esta forma:

```
{ "error": { "codigo": "SOLICITUD_INVALIDA", "mensaje": "...",
             "detalles": [{ "campo": "body.correo", "mensaje": "..." }] } }
```

Código HTTP	Código de error	Significado
400	SOLICITUD_INVALIDA	Datos inválidos (validación Zod).
401	NO_AUTENTICADO	Sin sesión o token vencido.
403	PROHIBIDO	El rol no tiene permiso.
404	NO_ENCONTRADO	El recurso no existe (o no es del usuario).
409	CONFLICTO	Fechas ocupadas, correo duplicado, recurso con dependencias.
422	REGLA_DE_NEGOCIO	Transición o regla no permitida.
429	CUENTA_BLOQUEADA / límite	Cuenta bloqueada o demasiados intentos.
503	SERVICIO_OCUPADO	La base de datos está bloqueada más de 5 s.

11.1 Autenticación

Método	Ruta	Acceso	Descripción
POST	/auth/registro	Público	`{ nombre, correo, contrasena }` → `{ token, usuario }`
POST	/auth/login	Público	`{ correo, contrasena }` → `{ token, usuario }`
POST	/auth/recuperar	Público	`{ correo }`; misma respuesta exista o no la cuenta
POST	/auth/restablecer	Público	`{ token, contrasena }` → sesión nueva
GET	/auth/yo	Sesión	Usuario actual

11.2 Catálogo y hoteles

Método	Ruta	Acceso	Descripción
GET	/catalogo/destinos, /catalogo/colecciones	Público	Con traducciones
GET	/catalogo/configuracion	Público	Tasa USD, IVA, limpieza, días de cancelación
GET	/hoteles?destino=&huespedes=&coleccion=	Público	Búsqueda sin tildes
GET	/hoteles/:id	Público	Detalle con imágenes, habitaciones y

Método	Ruta	Acceso	Descripción
			calificación
GET	/hoteles/:id/similares	Público	Prioriza el mismo destino
GET	/hoteles/:id/disponibilidad	Público	Disponibilidad por habitación
GET	/hoteles/:id/calendario?desde=&hasta=	Público	Noches ocupadas y temporadas (máx. 400 días)
GET	/hoteles/:id/cotizacion	Público	Precio noche por noche y política de cancelación
GET	/hoteles/:id/resenas	Público	Reseñas verificadas
POST / PATCH / DELETE	/hoteles[/:id]	Admin	CRUD de hoteles (409 si tiene reservas)
POST / PATCH / DELETE	/hoteles/:id/habitaciones[/:habitacionId]	Admin	CRUD de habitaciones (409 con reservas, 422 si es la última)

11.3 Reservas

Método	Ruta	Acceso	Descripción
POST	/reservas	Sesión	`{ hotelId, habitacionId, entrada, salida, huespedes }`
GET	/reservas/mias	Sesión	Reservas propias con cancelación, modificaciones y reseña
GET	/reservas?q=&estado=&hotelId=	Admin, recepción	Búsqueda por huésped, correo o código
GET	/reservas/exportar	Admin	CSV para Excel (`;`, UTF-8 con BOM, fórmulas neutralizadas)
GET	/reservas/:id	Propietario o personal	Detalle
PATCH	/reservas/:id	Admin, recepción	Modificar habitación, fechas o huéspedes
POST	/reservas/:id/check-in	Admin, recepción	Desde confirmada y desde el día de llegada
POST	/reservas/:id/check-out	Admin, recepción	Desde check-in
POST	/reservas/:id/cancelar	Propietario o personal	`{ exonerarCargo? }` (solo personal)

11.4 Reseñas, temporadas, administración e imágenes

Método	Ruta	Acceso	Descripción
POST	/resenas	Sesión	Puntajes 1–5 y comentario

Método	Ruta	Acceso	Descripción
GET / DELETE	/resenas[/:id]	Admin	Moderación
GET / POST / PATCH / DELETE	/temporadas[/:id]	Admin	`{ nombre, inicio, fin, recargo, hotelId }`
GET	/admin/accesos?dias=14	Admin	Estadísticas de inicio de sesión
GET	/admin/metricas?desde=&hasta=	Admin	Ocupación, ingresos, ADR, RevPAR (máx. 2 años)
POST	/imagenes	Admin	`multipart/form-data`, campo `imagen`
GET	/uploads/<nombre>	Público	Sirve la imagen (fuera de /api)
GET	/api, /api/salud	Público	Índice de rutas y estado del servidor

Ejemplo: crear una reserva

```
POST /api/reservas
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{ "hotelId": "mirador-del-cafe", "habitacionId": "cabana-finca",
  "entrada": "2026-12-05", "salida": "2026-12-08", "huespedes": 2 }

→ 201 { "datos": { "codigo": "VEL-699724", "estado": "confirmada",
                   "noches": 3, "subtotal": 2670000, "impuestos": 507300,
                   "tarifaLimpieza": 180000, "total": 3357300, ... } }
```

12. Seguridad

Amenaza	Mitigación implementada
Inyección SQL	Consultas preparadas con parámetros en todos los repositorios.
Robo de contraseñas	Hash bcrypt; los tokens de restablecimiento se guardan como SHA-256.
Sesiones robadas o antiguas	JWT con expiración de 8 h y versión de sesión verificada en cada petición.
Fuerza bruta	Bloqueo de cuenta (5 intentos / 15 min) y límite por IP en login, registro y recuperación (30 cada 15 min en producción).
Enumeración de cuentas	Mismos mensajes y conteo por correo exista o no la cuenta.
Manipulación de precios	El servidor calcula los montos; esquemas Zod estrictos rechazan campos extra.
Acceso a datos ajenos	Autorización por rol y por propietario; 404 para reservas ajenas.
XSS por archivos	Solo JPG, PNG y WebP verificados por contenido; SVG rechazado.
Fuga de metadatos	sharp elimina EXIF (incluida la ubicación GPS) al convertir a WebP.
Cabeceras y orígenes	Helmet, CORS con lista blanca, `x-powered-by` desactivado, cuerpo máximo de 100 kB.
Inyección de fórmulas en Excel	Las celdas del CSV exportado se neutralizan.
Datos personales	Purga diaria de registros de acceso con más de 90 días (Ley 1581 de 2012).

13. Frontend

13.1 Rutas de la aplicación

Ruta	Página	Acceso
/	Home	Público
/buscar	SearchResults	Público
/hotel/:id	HotelDetail	Público
/iniciar-sesion, /crear-cuenta, /recuperar	Login (tres modos)	Público
/restablecer	Restablecer	Público (con token)
/politicas	Politicas	Público
/mis-reservas	MisReservas	Rol cliente
/recepcion	RecepcionDashboard	Rol recepcionista
/admin	AdminDashboard (Resumen, Hoteles, Reservas, Temporadas, Reseñas, Accesos)	Rol admin
*	NoEncontrada	Público

Las páginas de cuenta, políticas, administración y recepción se cargan bajo demanda con `React.lazy` y `Suspense`, de modo que el sitio público descarga menos código. `RutaProtegida` redirige a quien no tenga el rol requerido.

13.2 Estado global y servicios

- **AuthContext**: sesión (token y usuario) guardada en `localStorage` con la clave `velora_sesion`; cierra la sesión si la API responde 401.
- **IdiomaContext**: idioma (ES/EN) y moneda (COP/USD); función `t()` para los textos de `i18n/textos.js`.
- **CatalogoContext**: destinos, colecciones y configuración pública cargados una vez.
- **services/api.js**: cliente `fetch` que adjunta el token, arma la URL (`VITE_API_URL`) y convierte los errores en `ErrorApi` con estado, código y detalles.
- **hooks/useConsulta.js**: carga de datos con estados de cargando, error y reintento.

13.3 Componentes principales

`Navbar`, `Footer`, `SearchBar`, `HotelCard`, `CalendarioDisponibilidad`, `ResenasHotel`, `Stars` y `EntradaEstrellas`, `ModificarReserva`, `Dialogo` (modal accesible), `SubirImagen`, `GraficaBarras`, `Imagen` (con `srcset`) y `Cargando`.

14. Servicios auxiliares

14.1 Correo electrónico

Se envía un correo al crear la cuenta, al confirmar, modificar o cancelar una reserva, al pedir un enlace de restablecimiento y cuando cambia la contraseña. Con `SMTP_HOST` se usa nodemailer; sin él (desarrollo), cada correo se guarda como `.html` en `data/correos/` y su ruta se muestra en la consola. En desarrollo, `POST /auth/recuperar` devuelve además `enlaceDesarrollo` para probar sin servidor de correo.

14.2 Imágenes

- Formatos admitidos: JPG, PNG y WebP de máximo 5 MB, comprobados por su contenido real.
- Cada imagen se convierte a WebP en tres tamaños: `<uuid>.webp` (1600 px), `<uuid>-800.webp` y `<uuid>-400.webp`; el sitio usa `srcset`.
- Se eliminan los metadatos y se respeta la orientación; imágenes de más de 40 megapíxeles o dañadas se rechazan con 400.

14.3 Tareas programadas

`server.js` ejecuta al arrancar y cada 24 horas la purga de registros de acceso con más de `ACCESOS_RETENCION_DIAS` días, y realiza un apagado ordenado (cierra la base de datos) al recibir SIGINT o SIGTERM.

15. Pruebas e integración continua

El backend cuenta con **74 pruebas automatizadas** (22 grupos) escritas con el ejecutor nativo `node:test`. Usan una base de datos en memoria, por lo que no alteran los datos reales. Se ejecutan con `cd backend && npm test`.

Archivo	Qué verifica
api.test.js	Autenticación, hoteles, reservas y robustez de la API.
concurrencia.test.js	Reservas simultáneas por la API, triggers y bloqueo entre conexiones.
funciones.test.js	Búsqueda, temporadas, calendario, cancelación, historial, reseñas, bloqueo, correos, recuperación, exportación, métricas y retención.
admin.test.js	Modificación de reservas y dashboard de accesos.
imagenes.test.js	Subida, validación y conversión de imágenes.
migracion.test.js	Migración de columnas nuevas en bases existentes.

El flujo `.github/workflows/pruebas.yml` se ejecuta en cada `push` y `pull request` con Node.js 24 y tiene dos trabajos: **Backend (pruebas)** (`npm ci` y `npm test`) y **Sitio (lint y compilación)** (`npm ci`, `npm run lint` y `npm run build`).

16. Despliegue a producción

1. Definir `NODE_ENV=production` en `backend/.env` (desactiva las cuentas demo y el enlace de desarrollo).
2. Definir un `JWT_SECRETO` largo y aleatorio (mínimo 32 caracteres).
3. Configurar `CORS_ORIGENES` y `URL_SITIO` con el dominio real del frontend.
4. Configurar el servidor SMTP (`SMTP_HOST`, `SMTP_PUERTO`, `SMTP_USUARIO`, `SMTP_CLAVE`, `CORREO_REMITENTE`).
5. Compilar el frontend con `npm run build` y publicar la carpeta `dist/` en un servidor web o CDN, con `VITE_API_URL` apuntando a la API.
6. Ejecutar el backend con `npm start` detrás de un proxy inverso con HTTPS (por ejemplo, Nginx) y un gestor de procesos (por ejemplo, PM2 o systemd).
7. Programar copias de seguridad periódicas de `data/velora.db` y de la carpeta `uploads/`.

17. Mantenimiento y solución de problemas

Problema	Causa probable	Solución
El servidor no arranca: "La base de datos ... es de una versión anterior"	Esquema viejo.	Ejecutar `npm run db:reset` (borra los datos) o migrar manualmente.
El servidor no arranca por una variable inválida	Error en `.env`.	Corregir el valor que indica el mensaje.
El sitio muestra "No pudimos cargar esta información"	Backend apagado o CORS.	Encender el backend y revisar `CORS_ORIGENES` y `VITE_API_URL`.
Error 503 SERVICIO_OCUPADO	Bloqueo de escritura prolongado.	Reintentar; revisar procesos que usen la base.
No llegan los correos	Sin SMTP configurado.	Revisar `data/correos/` o configurar SMTP.
`npm install` falla con sharp	Binario no compatible.	Usar Node.js 22.13+ y reinstalar.
Borrar usuarios de prueba	Datos de pruebas manuales.	`npm run db:borrar-pruebas -- --confirmar` (hace copia antes).

17.1 Cómo agregar una funcionalidad

1. Si cambia una tabla existente, modificar `esquema.sql` y subir `VERSION_ESQUEMA` en `db/conexion.js` (o agregar la columna en `COLUMNAS_NUEVAS`).
2. Crear o ampliar el módulo con sus capas: esquema Zod, repositorio, servicio y rutas.
3. Registrar la ruta en `app.js` y en el índice `INDICE_API`, y documentarla en `backend/README.md`.
4. Agregar pruebas en `backend/tests/` y ejecutar `npm test`.
5. Agregar el método en `Velora/src/services/api.js`, la interfaz y los textos en `i18n/textos.js` (español e inglés).
6. Ejecutar `npm run lint` y `npm run build` en `Velora/` antes de subir los cambios.

18. Trabajo futuro

- Pagos en línea (anticipo o pago total).
- Facturación electrónica ante la DIAN.
- Migración de la base de datos a PostgreSQL (por ejemplo, Supabase) para publicar el sitio con mayor concurrencia; el uso de SQL estándar, restricciones declarativas y triggers facilita la migración.